

Министерство образования и науки Российской Федерации
Санкт-Петербургский политехнический университет Петра Великого

Институт компьютерных наук и кибербезопасности
Высшая школа технологий искусственного интеллекта
Направление: 02.03.01 «Математика и компьютерные науки»

Отчёт о выполнении научно-исследовательской работы
На тему: «Развёртывание кластера SLURM и генерация потока
пользовательских заданий»

Выполнил студент
группы 5130201/40002

_____ Семенов И. А.

Проверил
преподаватель

_____ Чуватов М. В.

«_____» _____ 2026г.

Реферат

Отчёт посвящён развёртыванию вычислительного кластера под управлением SLURM¹ и разработке программы генерации пользовательских заданий на основе статистики реальной очереди. В ходе работы была подготовлена среда из пяти виртуальных машин на базе openSUSE Leap: управляющий узел и четыре вычислительных узла. Для узлов настроены статическая адресация, разрешение имён, SSH²-доступ по ключам, общая служба аутентификации MUNGE³ и конфигурация SLURM.

Для генерации потока заданий использован фрагмент набора данных SLURM, отфильтрованный по индивидуальному варианту: UID⁴ 50728 и имя задания qe7. По исходным данным рассчитаны распределения длительности выполнения и ошибки пользовательского прогноза времени. На основе полученных параметров реализован генератор пользовательских заявок SLURM, который формирует задания с командой `sleep`, задаёт плановое время выполнения и автоматически помещает задания в очередь. Результаты представлены через статистику выполненных заданий, графики исходных и сгенерированных распределений, а также сравнение планового и фактического времени выполнения.

Ключевые слова: SLURM, MUNGE, openSUSE Leap, кластер, виртуальная машина, пользовательская заявка, очередь заданий, логнормальное распределение, нормальное распределение, генерация заданий.

¹В современных реалиях используется название Slurm Workload Manager; исторически написание SLURM расшифровывалось как Simple Linux Utility for Resource Management.

²SSH — Secure Shell, протокол защищённого удалённого доступа.

³MUNGE — MUNGE Uid 'N' Gid Emporium, служба аутентификации, используемая SLURM для проверки сообщений между узлами.

⁴UID — User Identifier, числовой идентификатор пользователя в Unix-подобных системах.

Содержание

Реферат	2
Введение	4
1 Постановка задачи	5
2 Основная часть	6
2.1 Подготовка виртуальной среды	6
2.2 Настройка сети и доступа	6
2.3 Установка MUNGE и SLURM	7
2.4 Обработка исходного набора данных	9
2.5 Аппроксимация исходных распределений	10
2.6 Генерация заданий	13
2.7 Запуск заданий и сбор фактической статистики	14
2.8 Анализ фактической ошибки прогноза	17
Заключение	20
Список использованных источников	22
Приложение А. Пример сгенерированного задания SLURM	23
Приложение Б. Код генератора пользовательских заявок	24

Введение

Суперкомпьютерные системы используются для выполнения вычислительных задач, требующих значительного объёма процессорного времени и памяти. Такие системы обычно имеют кластерную архитектуру: множество вычислительных узлов объединены сетью и управляются специализированной системой диспетчеризации заданий. Одной из наиболее распространённых систем такого класса является SLURM.

Качество планирования в вычислительном кластере зависит от того, насколько точно пользователь оценивает требуемое время выполнения задания. На практике пользователь часто указывает время с запасом: фактическое время выполнения оказывается меньше запрошенного. Из-за этого диспетчер вынужден резервировать ресурсы на большее время, чем действительно необходимо, что усложняет планирование очереди и может снижать загрузку узлов.

Цель работы — сгенерировать поток пользовательских заданий SLURM на основе статистики реальной очереди и проверить его выполнение в кластере. Для этого необходимо построить кластер из виртуальных машин, настроить сетевое взаимодействие между узлами, подготовить конфигурацию SLURM, проанализировать исходный набор данных, разработать генератор заданий и сопоставить расчётные параметры с фактическими результатами запуска.

В отчёте описаны этапы подготовки виртуальных машин, настройки сетевой связности и SSH-доступа, установки MUNGE и SLURM, выбора параметров конфигурации, обработки исходного набора данных, генерации заданий и анализа результатов выполнения.

1 Постановка задачи

Необходимо сгенерировать поток пользовательских заданий на основе статистики реальной очереди SLURM, подготовить кластер для запуска этих заданий и проанализировать полученные результаты выполнения.

Задачи.

1. Создать пять виртуальных машин: один управляющий узел и четыре вычислительных узла.
2. Настроить статическую адресацию, разрешение имён и сетевую связность между всеми узлами.
3. Настроить SSH-доступ по ключам между узлами без ввода пароля.
4. Установить и настроить MUNGE для единой аутентификации.
5. Установить SLURM, сформировать конфигурационный файл через официальный конфигуратор и разложить его на все узлы.
6. Проверить состояние вычислительных узлов средствами SLURM.
7. Отфильтровать исходный набор данных по индивидуальному варианту.
8. Рассчитать параметры распределений длительности выполнения и ошибки прогноза времени по исходному набору данных.
9. Реализовать генератор пользовательских заявок SLURM.
10. Запустить сгенерированные задания на кластере и собрать статистику выполнения.
11. Представить результаты выполнения и сравнить исходные, сгенерированные и фактические характеристики заданий.

В варианте работы обозначены: дистрибутив openSUSE Leap, UID 50728, имя задания `qe7`. Исходные записи набора данных фильтруются по указанному UID и имени задания.

Результатом работы должны стать настроенный кластер SLURM, исходный код генератора, набор сгенерированных заданий, результаты выполнения на кластере, сравнение расчётных и фактических характеристик и отчёт, оформленный в соответствии с ГОСТ 7.32–2017.

2 Основная часть

2.1 Подготовка виртуальной среды

Практическая часть работы выполнялась на локальной машине с использованием Oracle VM VirtualBox⁵. В качестве модели вычислительного кластера была использована схема из пяти виртуальных машин: один управляющий узел и четыре вычислительных узла. Управляющий узел получил имя `mgmt`, вычислительные узлы получили имена `cn01`, `cn02`, `cn03` и `cn04`.

На все виртуальные машины был установлен дистрибутив `openSUSE Leap`⁶. При установке была выбрана минимальная консольная конфигурация без графической среды.

Каждой виртуальной машине был выделен один виртуальный процессор и 2 ГБ оперативной памяти. Первоначально для системного диска рассматривался объём 8–10 ГБ, однако установщик `openSUSE Leap` требовал не менее 12,5 ГиБ под корневой раздел и дополнительно 1–2 ГиБ под раздел подкачки. После этого виртуальные машины были пересозданы с диском объёмом 16 ГБ.

В результате была подготовлена однородная виртуальная среда: все узлы используют один дистрибутив, одинаковые аппаратные ограничения и минимальный набор служб. Сначала установка и базовая настройка были выполнены на управляющем узле, после чего тот же порядок действий был повторён на вычислительных узлах.

2.2 Настройка сети и доступа

Для каждой виртуальной машины использовались два сетевых адаптера. Первый адаптер был оставлен в режиме NAT⁷. Он нужен для доступа к внешним репозиториям `openSUSE` и установки пакетов через `zypper`⁸. Вторым адаптером был подключён к внутренней сети VirtualBox. Именно через этот адаптер организована локальная связь узлов кластера между собой.

Для внутренней сети был выбран диапазон IP-адресов `10.1.0.0/24`. Управляющему узлу назначен адрес `10.1.0.21`, вычислительным узлам на-

⁵Руководство пользователя Oracle VM VirtualBox: <https://www.virtualbox.org/manual/>.

⁶Документация `openSUSE Leap`: <https://doc.opensuse.org/>.

⁷NAT — Network Address Translation, механизм трансляции сетевых адресов.

⁸`zypper` — консольный менеджер пакетов `openSUSE`; справка по использованию: https://en.opensuse.org/SDB:Zypper_usage.

значены адреса 10.1.0.31, 10.1.0.32, 10.1.0.33 и 10.1.0.34. Адреса управляющего и вычислительных узлов разнесены по разным десяткам, поэтому конфигурация остаётся читаемой и допускает добавление новых узлов без изменения выбранной схемы.

Для доступа к виртуальным машинам с хостовой системы в VirtualBox были настроены пробросы SSH-портов через NAT: для `mgmt` использовался порт 2201, для `cn01–cn04` — порты 2202–2205. Это позволяло подключаться к каждому узлу с локальной машины, не открывая внешний доступ к внутренней сети кластера.

После настройки проброса портов на локальной машине была создана SSH-пара ключей и подготовлен пользовательский конфигурационный файл `~/.ssh/config`. В нём для каждого узла были указаны локальный адрес, соответствующий проброшенный порт и приватный ключ. После этого подключение с хостовой системы выполнялось по коротким именам узлов.

Разрешение имён выполнено через файл `/etc/hosts`. На всех узлах были добавлены одинаковые записи для имён `mgmt`, `cn01`, `cn02`, `cn03` и `cn04`.

Далее был настроен SSH-доступ без ввода пароля. Для этого на каждом узле была создана SSH-пара ключей, публичные ключи всех узлов были добавлены в файл `/etc/authorized/keys` на каждом узле. Дополнительно был создан пользовательский SSH-конфиг, в котором имена узлов сопоставлены с соответствующими хостами и выбранным приватным ключом. После настройки команда вида `ssh cn01` выполняется с любого узла без указания ключа и без ввода пароля.

2.3 Установка MUNGE и SLURM

Для работы SLURM требуется единый механизм аутентификации сообщений между узлами. Эту функцию выполняет MUNGE. На управляющем узле был создан общий ключ MUNGE, после чего он был скопирован на все вычислительные узлы. Для ключа были выставлены ограниченные права доступа и владелец `munge`; одинаковый ключ на всех узлах является условием корректного обмена служебными сообщениями SLURM.

Для корректной работы MUNGE также требуется согласованное системное время на всех узлах. При первоначальной установке на вычислительных узлах остался часовой пояс UTC+2, а на управляющем узле был уста-

новлен UTC+3. Из-за расхождения времени вычислительные узлы не могли корректно подключиться к кластеру: MUNGE проверяет время создания служебных сообщений и отклоняет сообщения, которые заметно отличаются от локального времени узла. После установки одинакового часового пояса и проверки текущего времени на всех виртуальных машинах службы MUNGE и SLURM начали взаимодействовать корректно.

Пакеты устанавливались через пакетный менеджер `zypper`. На управляющем узле используется служба `slurmctld`, которая хранит состояние очереди, принимает задания и назначает ресурсы. На вычислительных узлах используется служба `slurmd`, которая сообщает управляющему узлу о состоянии узла и выполняет назначенные задания.

Конфигурация SLURM была сформирована через официальный configurator SchedMD⁹. В конфигурации указан кластер `practice`, управляющий узел `mgmt`, вычислительные узлы `cn[01-04]` и очередь `debug`. Для выбора процесса отслеживания был оставлен вариант `proctrack/cgroup`; для выбора ресурсов использован `select/cons_tres`; для планировщика выбран `sched/backfill`. Эти параметры соответствуют рекомендациям configurator.

Один и тот же файл `slurm.conf` был размещён на всех узлах в каталоге `/etc/slurm`. На управляющем узле был создан каталог сохранения состояния контроллера, на вычислительных узлах был создан каталог для состояния `slurmd`. После запуска служб состояние кластера проверялось командами `sinfo` и `squeue`; перед запуском генерации вычислительные узлы должны находиться в состоянии `idle`.

Команда `sinfo` показывает состояние разделов SLURM и вычислительных узлов. Пример вывода для настроенного кластера:

```
PARTITION AVAIL  TIMELIMIT  NODES  STATE NODELIST
debug*      up    infinite      4    idle cn[01-04]
```

Команда `squeue` используется для просмотра очереди заданий; перед запуском генератора её пустой вывод означает, что в очереди нет активных или ожидающих заданий.

⁹Slurm Configuration Tool на сайте SchedMD: <https://slurm.schedmd.com/configurator.html>.

2.4 Обработка исходного набора данных

Индивидуальный вариант определяет три параметра: дистрибутив openSUSE Leap, UID 50728 и имя задания qe7. Исходный набор данных был распакован и отфильтрован по UID и имени задания. В результате сформирован файл `acct_0921-0923_uid50728_qe7`, содержащий 1563 записи.

В выбранном фрагменте 1542 записи имеют состояние `COMPLETED`, 11 записей имеют состояние `FAILED`, 10 записей имеют состояние `CANCELLED`. Для построения распределений использовались завершённые задания, так как только для них поле длительности выполнения можно интерпретировать как корректное время работы задачи.

Исходная длительность выполнения задания берётся из поля `ElapsedRaw`. В выгрузке SLURM поле `TimelimitRaw` задано в минутах, а `ElapsedRaw` — в секундах, поэтому при вычислении ошибки прогноза запрошенное время переводится в секунды:

$$\Delta T = \text{TimelimitRaw} \cdot 60 - \text{ElapsedRaw}.$$

В выбранном наборе данных минимальная длительность завершённого задания составляет 699 секунд, медиана составляет 1652 секунды, среднее значение составляет 1712.39 секунды. Максимальное значение равно 16660 секунд. Ошибка прогноза времени для завершённых заданий имеет медиану 19948 секунд и среднее значение 19887.61 секунды; следовательно, пользовательские задания в данном варианте обычно запрашивали время с большим запасом относительно длительности из исходной статистики.

На рисунке 1 приведена сводная визуализация отфильтрованной выборки. Она показывает, сколько записей осталось после отбора по индивидуальному варианту, какую долю составляют успешно завершённые задания. Эти данные используются дальше при подборе распределений для генератора.

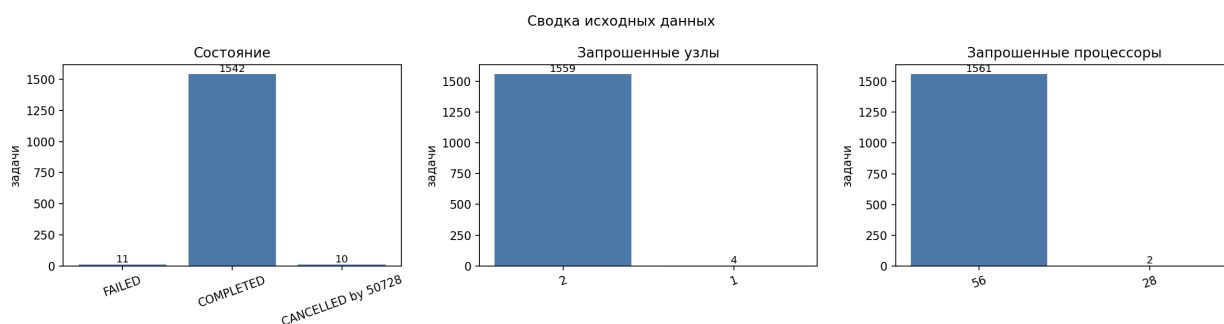


Рисунок 1. Сводка по исходному набору данных

2.5 Аппроксимация исходных распределений

Анализ исходных длительностей показал, что в данных присутствуют две выраженные группы заданий. Центр первой группы находится на уровне 1647 секунд, её вес равен 0.809; центр второй группы находится на уровне 1925 секунд, её вес равен 0.191. Поэтому длительность из исходного набора данных была описана смесью двух нормальных распределений. Параметры смеси приведены в таблице 1.

Таблица 1. Параметры смеси нормальных распределений исходной длительности

Компонента	μ , с	σ , с	Вес
1	1647.26	19.89	0.809
2	1924.82	18.70	0.191

Рисунок 2 показывает, что исходные длительности не образуют один общий пик: основная часть заданий сосредоточена вблизи 1650 секунд, меньшая группа — вблизи 1925 секунд. Наложенная кривая проходит через обе области концентрации, поэтому при генерации сохраняются два характерных режима выполнения, а не усреднённая длительность между ними.

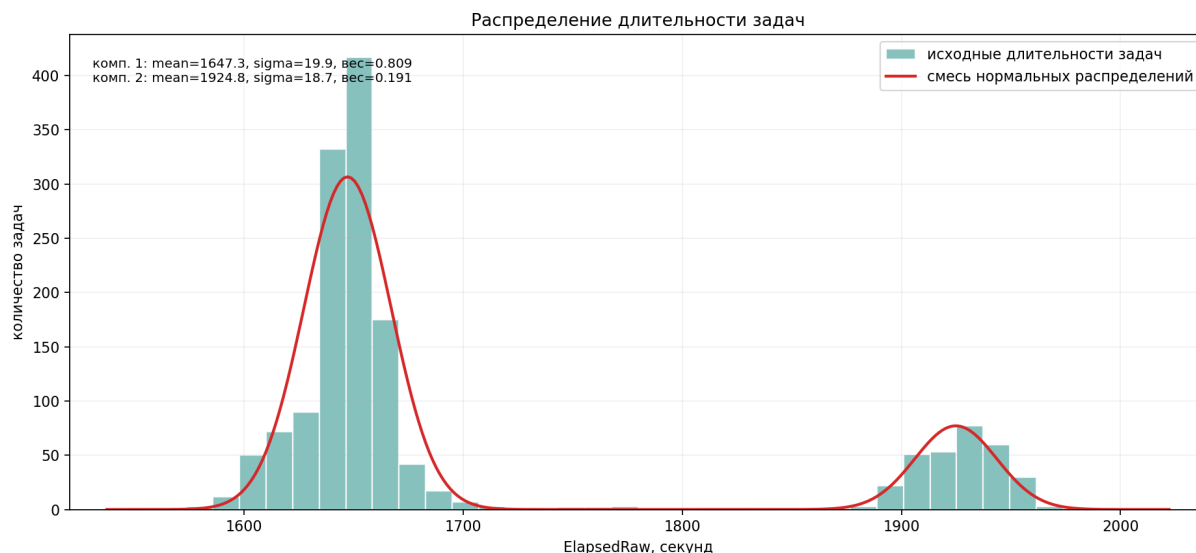


Рисунок 2. Аппроксимация распределения исходной длительности выполнения

В таблице 2 приведены параметры смеси логнормальных распределений для ошибки прогноза времени. Для каждой компоненты указаны параметры μ и σ в десятичной логарифмической шкале, а также вес компоненты в общей смеси. Первая компонента соответствует менее частой части выборки, вторая компонента является основной.

Таблица 2. Параметры смеси логнормальных распределений ошибки прогноза

Компонента	μ	σ	Вес
1	4.2939	0.0006	0.192
2	4.3000	0.0006	0.808

На рисунке 3 показан десятичный логарифм ошибки прогноза времени по исходному набору данных. Логарифмирование показывает, что основная масса значений разделяется на две близкие области; по ним заданы компоненты смеси из таблицы 2.

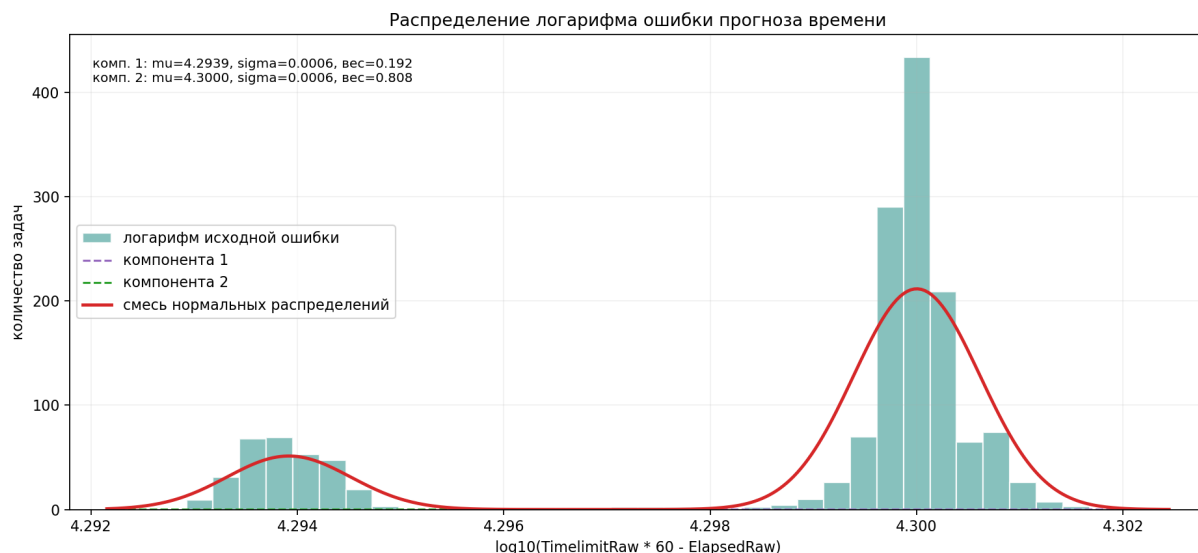


Рисунок 3. Распределение логарифма ошибки прогноза времени

На рисунке 4 показано распределение ошибки прогноза времени в масштабированной шкале запуска. Голубая гистограмма соответствует расчётной ошибке из манифеста.

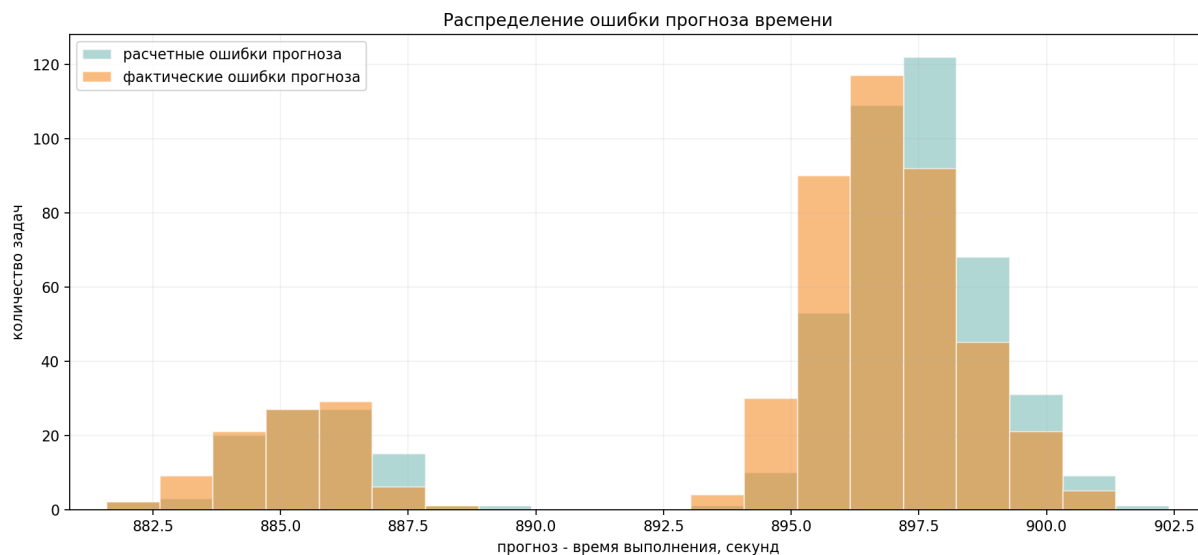


Рисунок 4. Распределение ошибки прогноза времени

На рисунке 4 также приведена фактическая ошибка после выполнения заданий в SLURM. Она показана оранжевой гистограммой и используется для сопоставления расчётной ошибки из манифеста с результатом запуска. Подробнее фактическая ошибка рассматривается в разделе 2.8.

2.6 Генерация заданий

Генератор запускается на управляющем узле `mgmt`. На вход ему передаются число заданий, масштаб длительности выполнения, масштаб планового времени и параметры распределений. Для каждого задания выбирается исходная длительность, ошибка прогноза и число требуемых узлов. Плановое время рассчитывается как сумма выбранной длительности и ошибки прогноза, после чего значения масштабируются для запуска на кластере.

Результатом работы генератора является каталог запуска с манифестом `manifest.csv` и набором файлов заданий `.slurm`. В манифест записываются имя файла задания, сгенерированные и масштабированные времена, лимит SLURM, число узлов и число задач. Каждый файл задания содержит параметры `#SBATCH`, команду `sleep` с рассчитанной длительностью и запись времени начала и окончания в общий журнал. Пример такого задания приведён в приложении А.

Масштабирование нужно для того, чтобы исходные задания длительностью в десятки минут можно было воспроизвести на кластере за ограниченное время. Масштаб длительности выполнения применяется к выбранной длительности задания и определяет аргумент команды `sleep`. Масштаб планового времени применяется к сумме выбранной длительности и ошибки прогноза; это значение записывается в манифест как плановое время и используется при расчёте ошибки прогноза. В параметре `--time` для SLURM передаётся то же плановое время, но округлённое до минут из-за формата лимита выполнения.

В манифесте для итогового запуска оба масштаба были выбраны 0.045. После масштабирования медиана времени `sleep` составила 74 секунды, среднее значение — 76.116 секунды, максимум — 89 секунд. Медиана планового времени составила 972 секунды. Так как длительность выполнения и плановое время масштабируются одинаково, соотношение между длительностью задания и запасом времени сохраняется.

В таблице 3 приведено распределение сгенерированных заданий по числу запрошенных узлов. Число узлов формируется по нормальному распределению и не привязано к значениям в исходной выборке. Большая часть заданий использует 2 узла, задания на 1 и 3 узла встречаются реже, задания на 4 узла составляют малую часть выборки.

Таблица 3. Распределение сгенерированных заданий по числу узлов

Число узлов	Число заданий
1	118
2	245
3	124
4	13

На рисунке 5 сравниваются исходное распределение длительностей и сгенерированная выборка. В сгенерированных данных сохраняются две основные группы заданий, при этом форма гистограммы отличается от исходной выборки из-за случайного выбора значений.

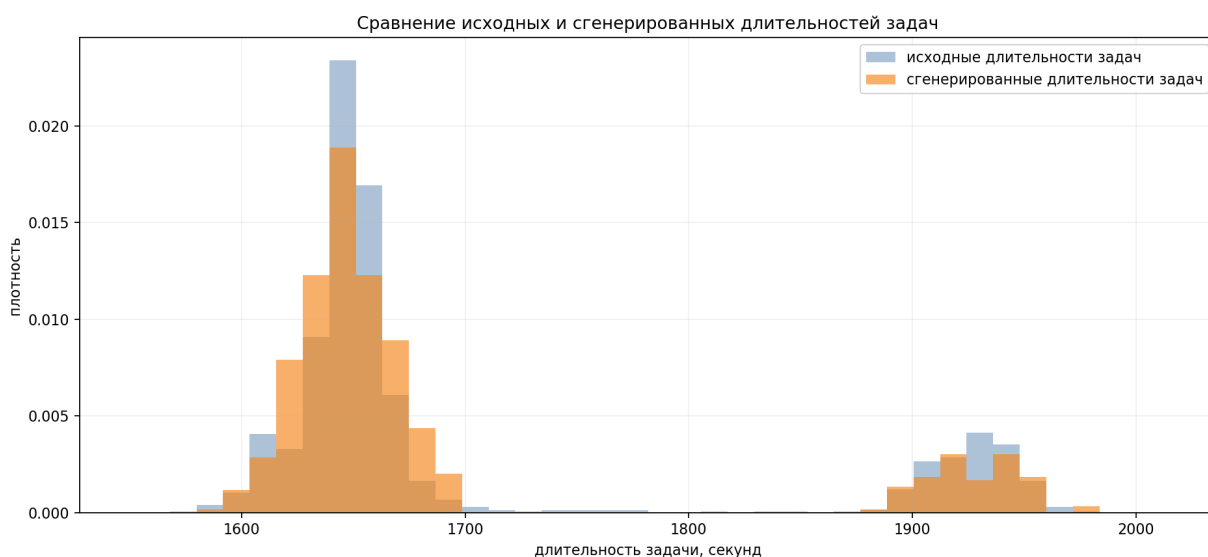


Рисунок 5. Сравнение исходной и сгенерированной длительности выполнения

2.7 Запуск заданий и сбор фактической статистики

После генерации задания отправляются в SLURM командой `sbatch`. Состояние очереди отслеживается через `squeue`, а сведения о завершённых заданиях сохраняются в файл `results.csv`. Для каждого задания фиксируются идентификатор SLURM, имя файла задания, состояние, узел выполнения, время ожидания в очереди, фактическое время выполнения и наблюдаемое полное время.

При анализе используются три временные величины. Запланированное время берётся из манифеста генератора. Значение `--time` передаётся в SLURM как лимит выполнения, но SLURM округляет его до минут. Фактическое время выполнения берётся из статистики SLURM. Поэтому фактическая ошибка прогноза считается по плановому значению из `manifest.csv`, а не по округлённому лимиту из очереди.

В итоговом запуске все 500 заданий завершились в состоянии COMPLETED. Запуск занял 6 часов 7 минут 4 секунды. Суммарное фактическое время выполнения всех заданий составило 10 часов 38 минут 28 секунд; оно больше времени всего запуска, так как задания выполнялись параллельно. Запланированное время `sleep` находилось в диапазоне от 72 до 89 секунд, медиана составила 74 секунды. Фактическое время выполнения находилось в диапазоне от 72 до 90 секунд, медиана составила 75 секунд. Медиана накладных расходов относительно команды `sleep` составила 0.562%, среднее значение — 0.656%, максимальное значение — 1.389%.

На рисунке 6 показано сравнение планового и фактического времени по отдельным заданиям.



Рисунок 6. Плановое и фактическое время выполнения заданий

Отклонение фактического времени по каждому заданию считается относительно времени `sleep`, записанного генератором в файл задания:

$$\delta_i = \frac{T_{\text{run},i}^{\text{SLURM}} - T_{\text{sleep},i}^{\text{manifest}}}{T_{\text{sleep},i}^{\text{manifest}}} \cdot 100\%.$$

Здесь $T_{\text{sleep},i}^{\text{manifest}}$ — запланированная длительность команды `sleep` для i -

го задания, а $T_{\text{run},i}^{\text{SLURM}}$ — фактическое время выполнения этого задания по статистике SLURM. На рисунке 7 показаны значения δ_i по всем заданиям. Они отражают накладные расходы запуска и завершения заданий в SLURM и виртуальной среде. В итоговом запуске все значения остаются ниже порога 3%.



Рисунок 7. Накладные расходы фактического выполнения по заданиям

Для сравнения был рассмотрен более короткий предварительный запуск на 200 заданий с масштабом времени 0.01, то есть с ускорением в 100 раз. В этом случае длительность `sleep` для большинства заданий попадает в диапазон 10–20 секунд, поэтому постоянные накладные расходы SLURM занимают заметную долю времени выполнения.



Рисунок 8. Накладные расходы при запуске 200 заданий с ускорением в 100 раз

На рисунке 8 часть значений выходит за контрольный порог 3%. В этом

запуске порог превысили 139 из 200 заданий, максимальное отклонение составило 12.5%, медианное значение — 5.0%. Поэтому для итогового запуска был выбран менее агрессивный масштаб времени: он увеличивает длительность отдельных заданий, но уменьшает относительный вклад накладных расходов.

2.8 Анализ фактической ошибки прогноза

Для удобства сопоставления на рисунке 9 повторно приведено распределение ошибки прогноза времени с наложением фактической ошибки после запуска заданий.

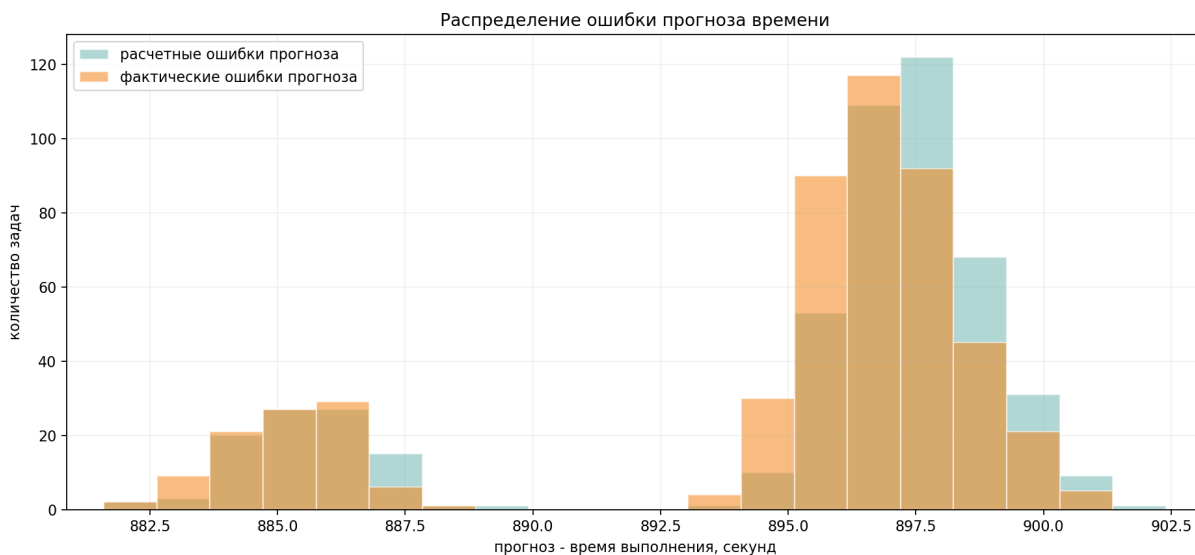


Рисунок 9. Сопоставление расчётной и фактической ошибки прогноза времени

Голубая гистограмма показывает запас времени, заложенный генератором до отправки заданий в очередь, оранжевая гистограмма показывает тот же запас после замены времени `sleep` на фактическое время выполнения из статистики SLURM. Распределения лежат в одном диапазоне, поэтому накладные расходы запуска не меняют характер ошибки прогноза, а только немного сдвигают отдельные значения.

После получения статистики SLURM ошибка прогноза пересчитывается для фактически выполненных заданий. Она определяется как разность между масштабированным плановым временем из манифеста и фактическим временем выполнения:

$$\Delta T_{\text{fact}} = T_{\text{plan}}^{\text{manifest}} - T_{\text{run}}^{\text{SLURM}}.$$

Здесь $T_{\text{plan}}^{\text{manifest}}$ — плановое время из манифеста генератора, а $T_{\text{run}}^{\text{SLURM}}$ — фактическое время выполнения, полученное из статистики SLURM.

На рисунке 10 показано распределение фактической ошибки прогноза. В запуске на 500 заданий ошибка находилась в диапазоне от 881 до 901 секунды, медиана составила 897 секунд. Наложенная кривая описывает наблюдаемую фактическую ошибку после выполнения заданий, а не параметры генератора.

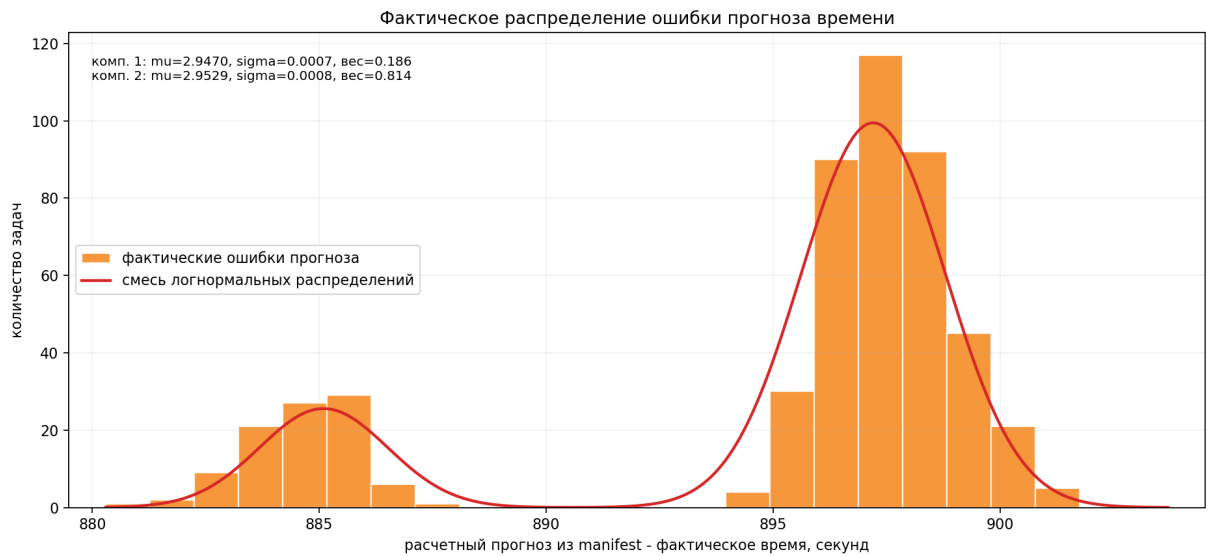


Рисунок 10. Распределение фактической ошибки прогноза времени

На рисунке 11 показан десятичный логарифм фактической ошибки прогноза. Фактические значения образуют компактную область рядом с распределением, заданным параметрами генератора. Небольшой сдвиг связан с тем, что фактическое время выполнения немного отличается от запланированного времени `sleep`.

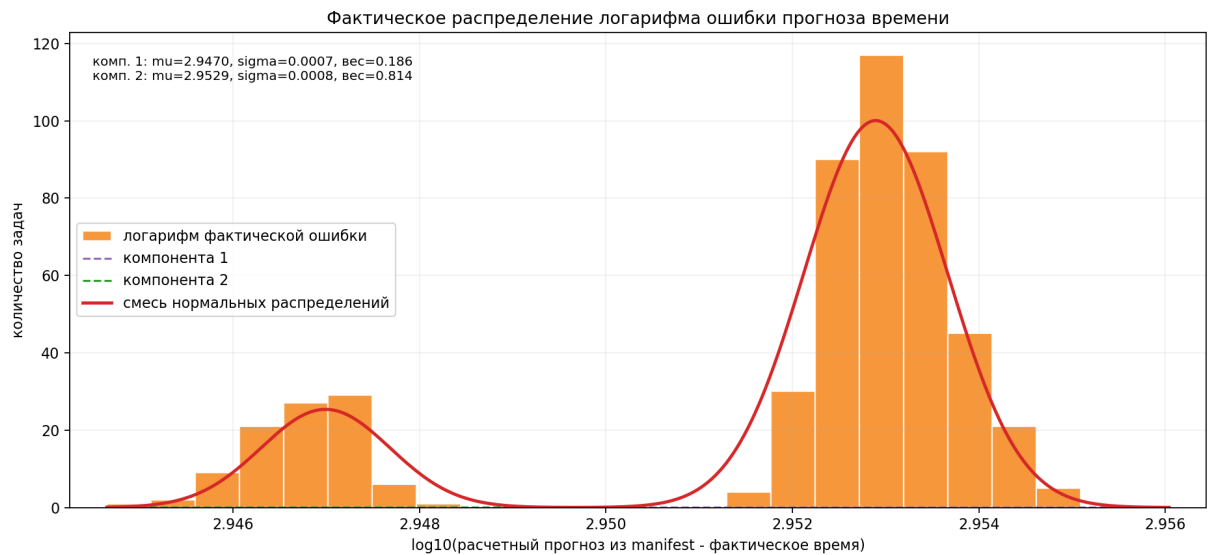


Рисунок 11. Распределение логарифма фактической ошибки прогноза времени

Итоговый запуск на 500 заданий показал, что выбранный масштаб времени подходит для проверки на четырёх вычислительных узлах. Медианное отклонение фактического времени от `sleep` составило 0.562%, среднее значение — 0.656%, максимальное значение — 1.389%. Фактическая ошибка прогноза остаётся близкой к расчётной ошибке из манифеста; это сопоставление показано на рисунке 9.

Заключение

В ходе работы был сгенерирован поток пользовательских заданий SLURM на основе статистики реальной очереди и подготовлена среда для проверки этого потока на вычислительном кластере. На локальной машине были созданы пять виртуальных машин с openSUSE Leap: управляющий узел `mgmt` и вычислительные узлы `cn01–cn04`. Для узлов настроены статические IP-адреса, разрешение имён, SSH-доступ по ключам и общая конфигурация MUNGE.

Запуск выполнялся на хостовой машине MacBook Air с процессором Apple M3, 8 вычислительными ядрами и 16 ГБ оперативной памяти под управлением macOS 26.5.1. Каждой виртуальной машине был выделен один виртуальный процессор, 2 ГБ оперативной памяти и виртуальный диск объёмом 16 ГБ.

По задачам, сформулированным в постановке, получены следующие результаты:

1. Создана схема из пяти виртуальных машин.
2. Настроены статическая адресация, разрешение имён и связность узлов.
3. Настроен SSH-доступ по ключам без ввода пароля.
4. Установлен MUNGE, подготовлен общий ключ и синхронизировано время.
5. Установлен и настроен SLURM.
6. Состояние вычислительных узлов проверено средствами SLURM.
7. Исходный набор данных отфильтрован по индивидуальному варианту.
8. Рассчитаны параметры распределений длительности выполнения и ошибки прогноза времени.
9. Реализован генератор пользовательских заявок SLURM.
10. Сгенерированные задания запущены на кластере, результаты выполнения собраны в журнал.
11. Исходные, сгенерированные и полученные при запуске характеристики сопоставлены в таблицах и на графиках.

Последний сформированный манифест содержит 500 заданий. При выбранном масштабе времени медианное плановое время составляет 972 секунды, медианное время `sleep` составляет 74 секунды. Итоговый запуск занял 6 часов 7 минут 4 секунды; медианное отклонение фактического времени от

`sleep` составило 0.562%, максимальное отклонение — 1.389%.

Таким образом, поставленная задача выполнена: кластер SLURM развёрнут, генератор пользовательских заявок реализован, исходные данные обработаны, а результаты генерации и выполнения представлены в виде численных характеристик и графиков.

Список использованных источников

- [1] Slurm Workload Manager Documentation [Электронный ресурс] // SchedMD. URL: <https://slurm.schedmd.com/documentation.html> (дата обращения: 21.06.2026).
- [2] Slurm Configuration Tool [Электронный ресурс] // SchedMD. URL: <https://slurm.schedmd.com/configurator.html> (дата обращения: 21.06.2026).
- [3] MUNGE Uid 'N' Gid Emporium [Электронный ресурс]. URL: <https://dun.github.io/munge/> (дата обращения: 21.06.2026).
- [4] openSUSE Leap Documentation [Электронный ресурс] // openSUSE. URL: <https://doc.opensuse.org/> (дата обращения: 18.06.2026).
- [5] ГОСТ 7.32–2017. Система стандартов по информации, библиотечному и издательскому делу. Отчёт о научно-исследовательской работе. Структура и правила оформления. М.: Стандартинформ, 2017.

Приложение А. Пример сгенерированного задания SLURM

```
1  #!/usr/bin/env bash
2  #SBATCH --job-name=ge7_0001
3  #SBATCH --partition=debug
4  #SBATCH --nodes=2
5  #SBATCH --ntasks=2
6  #SBATCH --time=00:17:00
7  #SBATCH --output=generated/logs/%x-%j.out
8  #SBATCH --error=generated/logs/%x-%j.err
9
10 set -euo pipefail
11
12 mkdir -p "$(dirname "generated/logs/generated-jobs.log")"
13 start_time="$(date --iso-8601=seconds)"
14 echo "job_index=1 source_job_id=507280001 start=${start_time} planned_sleep=74" >>
15   ↪ "generated/logs/generated-jobs.log"
16 sleep 74
17 end_time="$(date --iso-8601=seconds)"
18 echo "job_index=1 source_job_id=507280001 end=${end_time} planned_sleep=74" >>
19   ↪ "generated/logs/generated-jobs.log"
```

Приложение Б. Код генератора пользовательских заявок

Файл `generate/main.py` содержит разбор параметров, выбор значений из распределений, масштабирование времени и запись манифеста.

```
1  from __future__ import annotations
2
3  import argparse
4  import csv
5  import math
6  import random
7  import shutil
8  from pathlib import Path
9  from generate.template import render_batch_script
10
11
12
13  def positive_int(value: str) -> int:
14      parsed = int(value)
15      if parsed <= 0:
16          raise argparse.ArgumentTypeError("value must be positive")
17      return parsed
18
19
20  def positive_float(value: str) -> float:
21      parsed = float(value)
22      if parsed <= 0:
23          raise argparse.ArgumentTypeError("value must be positive")
24      return parsed
25
26
27  def parse_component(value: str) -> tuple[float, float, float]:
28      parts = value.split(",")
29      if len(parts) != 3:
30          raise argparse.ArgumentTypeError("component must be center,sigma,weight")
31      center = float(parts[0])
32      sigma = float(parts[1])
33      weight = float(parts[2])
34      if sigma <= 0 or weight <= 0:
35          raise argparse.ArgumentTypeError("sigma and weight must be positive")
36      return center, sigma, weight
37
38
39  def choose_component(rng: random.Random, components: list[tuple[float, float, float]]) ->
40      ↪ tuple[float, float]:
41      total = sum(weight for _, _, weight in components)
42      roll = rng.random() * total
43      cumulative = 0.0
44      for center, sigma, weight in components:
45          cumulative += weight
46          if roll <= cumulative:
47              return center, sigma
48      center, sigma, _ = components[-1]
49      return center, sigma
50
51  def sample_lognormal_value(rng: random.Random, components: list[tuple[float, float, float]])
52      ↪ -> int:
53      mu, sigma = choose_component(rng, components)
54      return max(1, int(round(rng.lognormvariate(mu, sigma))))
55
56  def sample_normal_value(rng: random.Random, components: list[tuple[float, float, float]]) ->
57      ↪ int:
58      mean, sigma = choose_component(rng, components)
59      return max(1, int(round(rng.normalvariate(mean, sigma))))
60
61  def sample_node_count(rng: random.Random) -> int:
62      sampled = int(round(rng.normalvariate(2.0, 0.75)))
63      return min(4, max(1, sampled))
64
65
66  def format_slurm_time(total_seconds: int) -> str:
```



```

67     total_seconds = max(60, int(math.ceil(total_seconds / 60.0) * 60))
68     days, remainder = divmod(total_seconds, 24 * 60 * 60)
69     hours, remainder = divmod(remainder, 60 * 60)
70     minutes, seconds = divmod(remainder, 60)
71     if days:
72         return f"{days}-{hours:02d}:{minutes:02d}:{seconds:02d}"
73     return f"{hours:02d}:{minutes:02d}:{seconds:02d}"
74
75
76 def clean_run_dir(run_dir: Path) -> None:
77     if run_dir.exists():
78         shutil.rmtree(run_dir)
79     (run_dir / "slurm_jobs").mkdir(parents=True)
80
81
82 def generate_jobs(
83     run_dir: Path,
84     count: int,
85     sleep_scale: float,
86     time_scale: float,
87     seed: int,
88     partition: str,
89     runtime_components: list[tuple[float, float, float]],
90     error_components: list[tuple[float, float, float]],
91 ) -> Path:
92     clean_run_dir(run_dir)
93     rng = random.Random(seed)
94     slurm_jobs_dir = run_dir / "slurm_jobs"
95     manifest_path = run_dir / "manifest.csv"
96     log_file = Path("generated/logs/generated-jobs.log")
97
98     with manifest_path.open("w", newline="", encoding="utf-8") as manifest_file:
99         writer = csv.DictWriter(
100             manifest_file,
101             fieldnames=[
102                 "script",
103                 "source_job_id",
104                 "source_state",
105                 "source_elapsed_seconds",
106                 "sampled_elapsed_seconds",
107                 "sampled_error_seconds",
108                 "generated_forecast_seconds",
109                 "scaled_sleep_seconds",
110                 "scaled_forecast_seconds",
111                 "slurm_time",
112                 "nodes",
113                 "ntasks",
114             ],
115         )
116         writer.writeheader()
117
118         for index in range(1, count + 1):
119             sampled_elapsed_seconds = sample_normal_value(
120                 rng, runtime_components
121             )
122             sampled_error_seconds = sample_lognormal_value(rng, error_components)
123             generated_forecast_seconds = sampled_elapsed_seconds + sampled_error_seconds
124             scaled_sleep_seconds = max(1, int(round(sampled_elapsed_seconds * sleep_scale)))
125             scaled_forecast_seconds = max(
126                 60,
127                 int(round(generated_forecast_seconds * time_scale)),
128             )
129             nodes = sample_node_count(rng)
130             ntasks = nodes
131             slurm_time = format_slurm_time(scaled_forecast_seconds)
132             source_job_id = str(507280000 + index)
133             script_name = f"job_{index:04d}_{source_job_id}.slurm"
134             script_path = slurm_jobs_dir / script_name
135
136             script_path.write_text(
137                 render_batch_script(
138                     index=index,
139                     source_job_id=source_job_id,
140                     partition=partition,
141                     nodes=nodes,
142                     ntasks=ntasks,
143                     slurm_time=slurm_time,
144                     sleep_seconds=scaled_sleep_seconds,

```

```

145         log_file=log_file,
146     ),
147     encoding="utf-8",
148 )
149 script_path.chmod(0o755)
150
151 writer.writerow(
152     {
153         "script": script_name,
154         "source_job_id": source_job_id,
155         "source_state": "GENERATED",
156         "source_elapsed_seconds": sampled_elapsed_seconds,
157         "sampled_elapsed_seconds": sampled_elapsed_seconds,
158         "sampled_error_seconds": sampled_error_seconds,
159         "generated_forecast_seconds": generated_forecast_seconds,
160         "scaled_sleep_seconds": scaled_sleep_seconds,
161         "scaled_forecast_seconds": scaled_forecast_seconds,
162         "slurm_time": slurm_time,
163         "nodes": nodes,
164         "ntasks": ntasks,
165     }
166 )
167
168 return manifest_path
169
170
171 def parse_args() -> argparse.Namespace:
172     parser = argparse.ArgumentParser(description="Generate Slurm jobs on the mgmt VM.")
173     parser.add_argument("--output-root", type=Path, default=Path("generated/cluster_runs"))
174     parser.add_argument("--run-id", required=True)
175     parser.add_argument("--count", type=positive_int, default=200)
176     parser.add_argument("--sleep-scale", type=positive_float, default=0.01)
177     parser.add_argument("--time-scale", type=positive_float, default=0.01)
178     parser.add_argument("--seed", type=int, default=50728)
179     parser.add_argument("--partition", default="debug")
180     parser.add_argument(
181         "--runtime-normal-component",
182         action="append",
183         type=parse_component,
184         default=[],
185         help="Runtime normal component as mean,sigma,weight.",
186     )
187     parser.add_argument(
188         "--error-component",
189         action="append",
190         type=parse_component,
191         default=[],
192         help="Forecast error lognormal component as mu,sigma,weight.",
193     )
194     return parser.parse_args()
195
196
197 def main() -> None:
198     args = parse_args()
199     run_dir = (args.output_root / args.run_id).resolve()
200     runtime_components = args.runtime_normal_component or [(1701.49, 435.21, 1.0)]
201     error_components = args.error_component or [(9.8981, 0.0371, 1.0)]
202     generate_jobs(
203         run_dir=run_dir,
204         count=args.count,
205         sleep_scale=args.sleep_scale,
206         time_scale=args.time_scale,
207         seed=args.seed,
208         partition=args.partition,
209         runtime_components=runtime_components,
210         error_components=error_components,
211     )
212     print(run_dir)
213
214
215 if __name__ == "__main__":
216     main()

```

Файл `generate/template.py` формирует текст задания SLURM.

```
1 from __future__ import annotations
```

```

2
3 from pathlib import Path
4
5
6 def render_batch_script(
7     index: int,
8     source_job_id: str,
9     partition: str,
10    nodes: int,
11    ntasks: int,
12    slurm_time: str,
13    sleep_seconds: int,
14    log_file: Path,
15 ) -> str:
16     return f"""#!/usr/bin/env bash
17 #SBATCH --job-name=qe7_{index:04d}
18 #SBATCH --partition={partition}
19 #SBATCH --nodes={nodes}
20 #SBATCH --ntasks={ntasks}
21 #SBATCH --time={slurm_time}
22 #SBATCH --output=generated/logs/%x-%j.out
23 #SBATCH --error=generated/logs/%x-%j.err
24
25 set -euo pipefail
26
27 mkdir -p "$(dirname "{log_file}")"
28 start_time="$(date --iso-8601=seconds)"
29 echo "job_index={index} source_job_id={source_job_id} start=${{start_time}}
   ↳ planned_sleep={sleep_seconds}" >> "{log_file}"
30 sleep {sleep_seconds}
31 end_time="$(date --iso-8601=seconds)"
32 echo "job_index={index} source_job_id={source_job_id} end=${{end_time}}
   ↳ planned_sleep={sleep_seconds}" >> "{log_file}"
33 """

```